BCM vs API Transformer vs FANTF

A controlled comparison on the same volatility series, chronological split, and predict-then-learn evaluation protocol.

FANTF records the best learned-model result: RMSE 1.4063. It reduces RMSE by 34.3% versus BCM and 1.4% versus the API Transformer.

Executive findings

BCM replacement	Both Transformer variants materially outperform the reproduced BCM path. FANTF lowers RMSE from 2.1420 to 1.4063 and MAE from 1.7207 to 1.0511.
FANTF versus API Transformer	FANTF improves RMSE by 1.4% and MAE by 2.3%. This is a small single-seed advantage, so it should be treated as preliminary.
Baseline context	Persistence remains stronger on test RMSE (1.3727) than FANTF (1.4063). The evidence supports FANTF as the strongest learned replacement here, but not as the best forecast overall.

Test results

Metric	BCM	API Transformer	FANTF	Best
RMSE	2.1420	1.4263	1.4063	FANTF
MAE	1.7207	1.0754	1.0511	FANTF
Pearson r	0.9044	0.9450	0.9470	FANTF
Predictive R-squared	0.7453	0.8870	0.8902	FANTF

All values use the same 374 held-out examples. Lower RMSE and MAE are better; higher correlation and predictive R-squared are better.

1. Experimental design

Item	Shared setting
Data	860 next-step windows from otmVolatility.mat; five observed values predict the next value.
Chronology	486 warmup examples followed by 374 test examples; first scored MATLAB window is 487.
Evaluation	Prequential: predict first, reveal the target, then update with a 120-example replay window.
Transformer settings	One encoder layer, two heads, d_model = 8, feed-forward width = 32, AdamW, 120 warmup epochs, seed 42.
Normalization	Fitted only on observations available before the first scored target.

2. What differs between the models

Model	Core mechanism	Complexity record
BCM	The reproduced ieRSPOP++ MATLAB path updates fuzzy rule associations online and combines its rule outputs into a forecast.	Reported weighted rule count: 227.95.
API Transformer	The archived project model uses the PyTorch Transformer encoder API with a persistence residual: latest value plus a learned correction.	953 trainable parameters.
FANTF	Custom Q/K/V attention adds delta * alpha * Gaussian noise to query-key scores before softmax. Alpha is learned per encoder layer; delta = 0.5.	954 trainable parameters.

The API label refers to the built-in PyTorch Transformer programming interface used by the earlier project model. It does not refer to a hosted model or an external prediction service. FANTF keeps the same forecasting head and evaluation protocol, while moving fuzzy behavior inside each custom attention layer.

3. Validation-period check

Model	Validation RMSE	Validation MAE
API Transformer	1.6330	1.2212
FANTF	1.5832	1.1510

BCM has no matching saved validation-period artifact, so this secondary table compares only the two Transformer variants.

4. Quantitative comparison

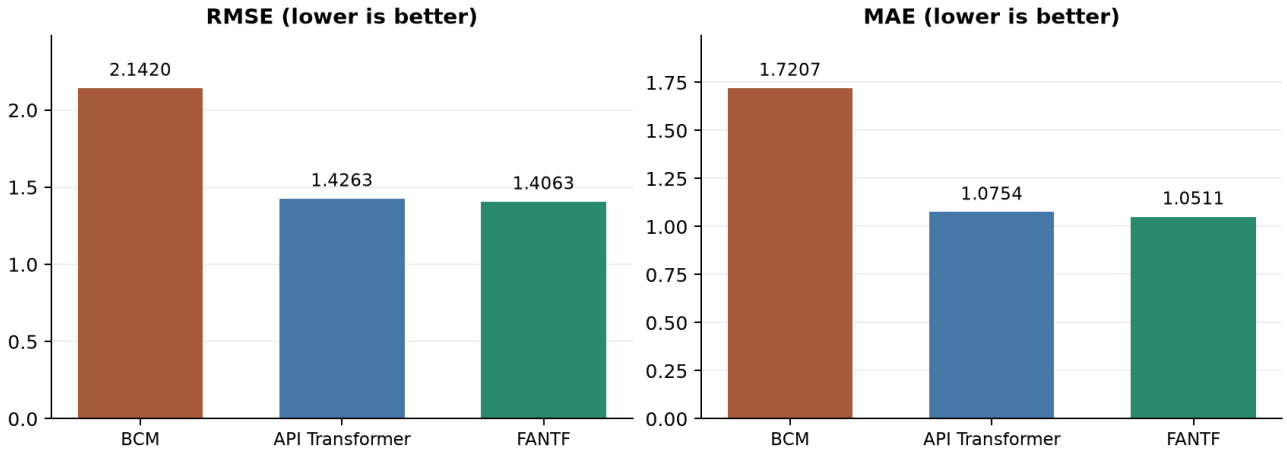


Figure 1. FANTF is slightly ahead of the API Transformer on both error measures and substantially ahead of BCM.

Comparison	RMSE change	MAE change	Interpretation
FANTF vs BCM	34.3% lower	38.9% lower	Large improvement
API Transformer vs BCM	33.4% lower	37.5% lower	Large improvement
FANTF vs API Transformer	1.4% lower	2.3% lower	Small improvement

5. Forecast behavior

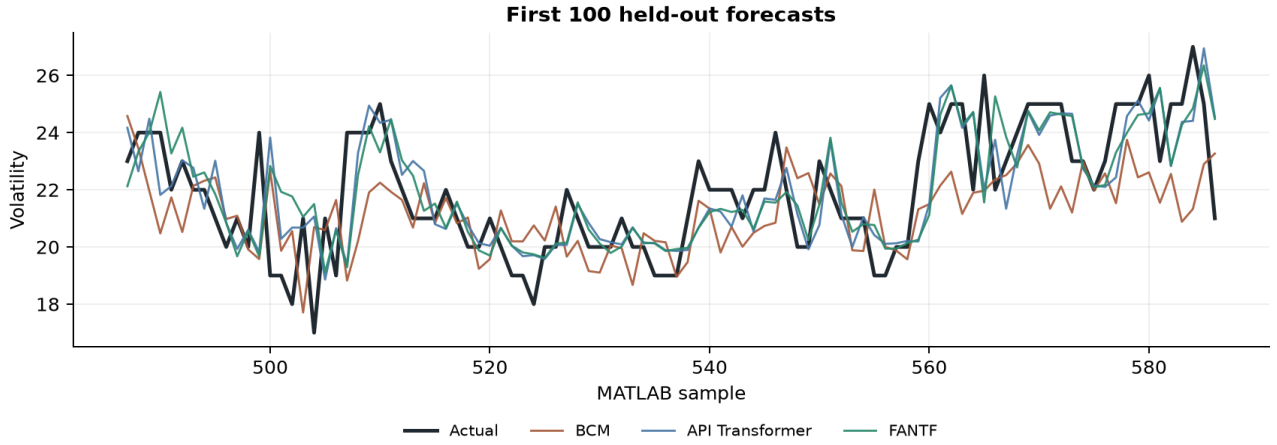


Figure 2. A zoomed view of the first 100 test cases. The two Transformer forecasts generally track one another more closely than either tracks BCM.

6. Error accumulation

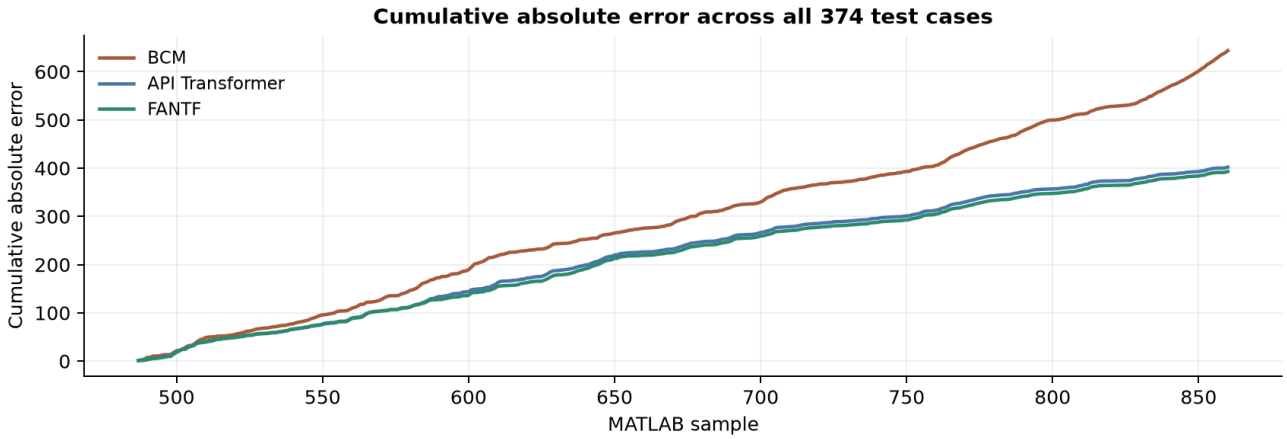


Figure 3. Lower cumulative absolute error is better. FANTF finishes slightly below the API Transformer, while BCM accumulates error faster.

7. Interpretation

Supported conclusion	FANTF is the strongest of the three learned systems in this recorded run, and both Transformer variants are much stronger than the reproduced BCM path.
Strength of FANTF claim	The difference from the API Transformer is modest. One deterministic seed is enough for a reproducible comparison, but not enough to establish a stable statistical advantage.
Forecasting ceiling	Persistence RMSE is 1.3727, which is 2.4% lower than FANTF. This simple baseline must remain in future comparisons.
Recommended next experiment	Run at least five seeds for the API Transformer and FANTF, report mean and standard deviation, and tune delta only on chronological validation folds.

8. Reproducibility notes and limitations

The BCM result was reproduced through the archived MATLAB online training and error calculation path. Local compatibility implementations of gauss2mf, mapminmax, and corr2 replace unavailable toolbox functions; this should accompany any BCM claim. The API Transformer result comes from the archived project artifact produced before the custom attention rewrite. The FANTF result uses Gaussian score perturbations during both training and prediction, with seed 42 fixing the reported path. Model parameter count and BCM rule count describe different kinds of complexity and should not be compared as equivalent units.

Appendix A. Result provenance

Result	Artifact
BCM metrics	outputs/bcm_matlab/metrics.json
BCM predictions	outputs/bcm_matlab/predictions.csv
API Transformer metrics	outputs/transformer_residual_d_model_8/metrics.json
API Transformer predictions	outputs/transformer_residual_d_model_8/predictions.csv
FANTF metrics	outputs/fantf_style/metrics.json
FANTF predictions	outputs/fantf_style/predictions.csv

Appendix B. External references

1	Chakraborty, S. and Heintz, F. (2025). Enhancing Time Series Forecasting with Fuzzy Attention-Integrated Transformers. arXiv:2504.00070. https://arxiv.org/abs/2504.00070
2	FANTF reference implementation. https://github.com/sanjaylopa22/FANTF
3	Vaswani, A. et al. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems 30.

Appendix C. Configuration snapshot

Setting	API Transformer	FANTF
Sequence length	5	5
Encoder layers / heads	1 / 2	1 / 2
d_model / feed-forward	8 / 32	8 / 32
Warmup epochs	120	120
Learning rate	0.001	0.001
Replay window	120	120
Persistence residual	Enabled	Enabled
Fuzzy attention	Disabled	Enabled; delta = 0.5
Mamdani post-layer rules	Disabled	Disabled
Seed	42	42